PayTrack: de OLTP a OLAP

Lab 1 · Modelado dimensional | 22 de octubre de 2026

1. Diseño y decisiones

Proceso: procesamiento de pagos. **Grano:** una fila por transacción, aprobada o rechazada. transaction_id es la clave primaria y permite rastrear el pago. Cinco claves foráneas obligatorias lo conectan con sus dimensiones.

Dimensión	Atributos y función
Cliente (SCD2)	customer_key; customer_id, ciudad y segmento; valid_from, valid_to e is_current. Cada versión tiene su propia clave.
Comercio	merchant_key; merchant_id, nombre, categoría y ciudad.
Método de pago	payment_method_key; method_name y method_type: card, digital_wallet o bank_transfer; other para futuros métodos.
Fecha y hora	date_key (YYYYMMDD): fecha, año, trimestre, mes, semana, día y banderas. time_key (HHMM): hora, minuto y franja. Se conservan las dimensiones dadas.

Datos excluidos: nombre y correo del cliente, ingreso exacto y fecha de alta: no son necesarios para estas preguntas. El identificador natural se conserva para conciliar la carga; sigue siendo un identificador que debe protegerse. No se lleva el detalle de eventos de fraude a esta tabla de hechos: cambiaría el grano.

Medidas: amount_cop y los conteos de pagos, fraudes y aprobaciones son aditivos. Los booleanos se suman mediante CASE o se cuentan con filtros. El promedio, la tasa de fraude y fraud_risk_score son no aditivos; la tasa se recalcula con sus numeradores y denominadores. No hay medidas semiaditivas en este modelo.

Surrogate keys: las claves enteras de negocio permiten varias versiones de un cliente y desacoplan el DW de los identificadores de origen. Las llaves naturales se conservan para trazabilidad. Fecha y hora mantienen las claves determinísticas del contrato.

SCD2: segmento y ciudad pueden cambiar. Sobrescribir el segmento reasignaría pagos antiguos al segmento actual y alteraría su tasa histórica de fraude. El hecho se vincula por el intervalo [valid_from, valid_to); Q2 no filtra is_current. Un índice único parcial impide dos versiones vigentes, y se validan los límites de cada intervalo.

Alcance: este laboratorio hace una carga inicial completa, no un proceso incremental. Como la fuente no contiene historia, la primera versión usa valid_from = -infinity y valid_to = NULL. Esto aplica el snapshot disponible a los pagos existentes; no recupera cambios históricos desconocidos. Una carga incremental futura debe cerrar y abrir versiones sin solapamientos y conservar sus claves.

Diagrama completo editable: esquema_estrella.drawio. La vista exportada con draw.io está dentro del notebook. La bandera is_holiday_co queda en FALSE como en el cargador dado; no representa un calendario oficial de festivos.

Planes y rendimiento

2. Diagnóstico OLTP y comparación

Método: PostgreSQL 16.15 en Docker; zona America/Bogota; corte CURRENT_DATE = 2026-10-22. Datos sintéticos de 90 días (24-jul a 21-oct), semilla 20261022, 500 clientes y 8 comercios. El lote de 50K es un subconjunto del de 500K. Se ejecutó VACUUM (ANALYZE) en ambos modelos, dos calentamientos y siete mediciones por consulta/configuración, alternando el orden de Q1-Q3. Se reporta la mediana de Execution Time de EXPLAIN (ANALYZE, BUFFERS); no incluye la transferencia al cliente.

Consulta	OLTP 50K ms	OLAP 50K ms	OLTP 500K ms	OLAP 500K ms
Q1 Categoría	6.63	6.13	39.09	57.26
Q2 Fraude	13.51	13.63	61.99	53.75
Q3 Día / hora	3.91	1.90	39.75	14.07

OLAP usa el índice parcial de fecha con columnas incluidas. Mismo servidor y configuración; shared_buffers = 128 MB, work_mem = 4 MB, hasta 2 workers por Gather. Las mediciones son con caché caliente y las configuraciones se recorrieron por bloques; no representan una carga concurrente de producción.

Q1 - monto por categoría. En 50K, OLTP usa Bitmap Heap/Index Scan; en 500K usa Parallel Seq Scan, Hash Join con comercios y agregación parcial. La 3FN obliga a buscar la categoría en merchants. En 500K examina los 500.000 pagos y conserva 166.817. OLAP usa Nested Loop con 30 fechas, Index Only Scan sobre hechos y Hash Join con comercio. Lee solo los pagos elegibles y registra 915 accesos a bloques frente a 5.707 de OLTP, pero su plan serial tarda más: 57.26 frente a 39.09 ms. Menos accesos no garantiza menor latencia.

Q2 - fraude por segmento. Ambos modelos usan scans completos, Hash Join y agregaciones; en 500K los scans son paralelos. La 3FN mantiene el segmento separado de las transacciones. Los dos modelos deben procesar los 500.000 pagos porque no hay filtro. OLAP emplea claves enteras y una dimensión más estrecha; en esta prueba reduce bloques de 5.728 a 5.214 y tiempo de 61.99 a 53.75 ms. La diferencia no aísla por sí sola la causa.

Q3 - día y hora. OLTP usa Bitmap Heap/Index Scan y calcula EXTRACT por pago; esta consulta no necesita joins, por lo que su costo no se debe a la normalización. En 500K procesa 38.963 pagos elegibles. OLAP lee esos pagos por Index Only Scan, une fecha mediante Nested Loop y hora mediante Hash Join; agrupa atributos ya calculados. El tiempo baja de 39.75 a 14.07 ms, con 224 accesos a bloques frente a 5.800.

Operaciones dominantes: lectura, joins y agregación. Los sorts finales trabajan con 7 categorías, 3 segmentos o 168 combinaciones; en estos planes no son el cuello de botella. Los planes del notebook incluyen filas por nodo y loops; PostgreSQL redondea promedios cuando hay varias iteraciones.

Índices, escala y validación

3. Experimento y límites de la conclusión

Índices en hechos (500K)	Q1 ms	Q2 ms	Q3 ms
Solo PK (sin índices secundarios)	45.72	55.41	32.19
Fecha parcial + INCLUDE	57.26	53.75	14.07
Solo customer_key	44.95	53.91	31.66
Fecha + customer_key	56.96	58.70	14.12

Índice conservado: date_key con INCLUDE (merchant_key, time_key, amount_cop), WHERE is_approved. Cubre Q1 y Q3. Tras VACUUM, ambos planes muestran Heap Fetches = 0. La ventana de 7 días selecciona 7,79% del lote: el índice reduce Q3 de 32.19 a 14.07 ms. En Q1 (33,36% del lote), el plan serial elegido con índice es más lento que el scan paralelo sin él. Se conserva por Q3, reconociendo ese costo para Q1.

Índice descartado: customer_key. Q2 necesita todas las filas y el optimizador conserva el mismo scan completo; Q1 y Q3 tampoco lo usan. Las pequeñas variaciones de tiempo con el mismo plan no prueban un beneficio. El archivo final evita este costo de almacenamiento y mantenimiento. Los cuatro escenarios también están medidos en 50K.

¿Por qué sequential scan? Leer buena parte de una tabla de forma secuencial puede costar menos que recorrer un índice y visitar páginas dispersas. El optimizador estima selectividad, costos de acceso y paralelismo; un índice disponible no obliga a usarlo. Las estadísticas actualizadas ayudan, pero el plan de menor costo estimado no siempre gana en tiempo real [1, 2].

Escala: 500.000 pagos diarios durante 365 días equivalen a **182,5 millones de filas**, 365 veces el lote mayor. Con el mismo ritmo, 30 días serían 15 millones y 7 días, 3,5 millones. Q2 crecería con toda la historia; las ventanas de Q1/Q3 seguirían siendo grandes. No es válido multiplicar estos milisegundos linealmente: cambian caché, I/O, paralelismo y planes. Al pasar de 50K a 500K, Q1 crece 5,90x en OLTP y 9,34x en OLAP; Q2, 4,59x y 3,94x; Q3, 10,18x y 7,39x. Particionar por fecha y mantener agregados serían experimentos posteriores, no resultados demostrados aquí.

4. Carga y controles

Los cuatro INSERT ... SELECT transforman en PostgreSQL. Se traducen claves, se evalúa la vigencia del cliente y se hace LEFT JOIN con eventos preagrupados por transacción: MAX(risk_score) conserva el mayor riesgo y evita multiplicar hechos. Si no hay evento, el riesgo queda NULL.

Resultado: las 9 validaciones del cargador y las 3/3 consultas pasan en ambos tamaños. En 500K: **500.000 hechos, $37.301.067.299,61 COP, 1.913 fraudes y 499.429 aprobados**. Además, se comprobó igualdad exacta con Decimal, sin el redondeo del evaluador. Tres pruebas pequeñas cubren montos, eventos múltiples, fechas límite y un cambio SCD2 con resultado esperado calculado a mano. Los validadores dados se conservaron.

Conclusión: el modelo facilita el análisis histórico y acelera Q2/Q3 en 500K; no mejora todas las consultas automáticamente. La evidencia justifica el diseño dimensional y la selección de índices, sin prometer una ventaja universal.

Evidencia: resultados.csv conserva las siete mediciones por caso. Los planes completos y las validaciones están en Lab1_PayTrack.ipynb. Los planes del anexo corresponden a ejecuciones adicionales; sus tiempos pueden diferir de las medianas.

[1] PostgreSQL 16: EXPLAIN. [2] PostgreSQL 16: Examining Index Usage.